UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

Escuela Profesional de Ingeniería Estadística e Informática



Universidad
Nacional del
Altiplano de Puno



CAPÍTULO I

SOLUCIÓN DE EJERCICIOS DE PROGRAMACIÓN

Asignatura: Estructura de Datos

Docente:	Fred Torrez Cruz
Estudiante:	Alberto Roy Chambilla Murillo
Semestre Académico:	2026-I
Lugar y Fecha:	Puno, Perú — 2026
Repositorio GitHub:	Ver código fuente en GitHub

Presentación

El presente documento contiene la solución detallada y estructurada de los ejercicios prácticos del **Capítulo I** para el curso de **Estructura de Datos**. Cada ejercicio incluye la justificación metodológica, la implementación del código fuente en lenguaje C/C++ y la referencia a su correspondiente ejecución gráfica.

Puedes acceder al código fuente completo y actualizado en el siguiente repositorio de GitHub:

https://github.com/albertoroy3210-creator/Ejercicios-de-estructura/blob/main/Codigo/Pdf/Latex/Ejercicios%20Cap.1.tex

---

Ejercicio 1. Costo total del procesamiento

Procedimiento: Se leen las horas trabajadas y el costo por hora. El costo total se obtiene multiplicando ambos valores.

```c
#include <stdio.h>

int main(void) {
    float horas, tarifa, costo;

    printf("Ingrese las horas trabajadas: ");
    scanf("%f", &horas);

    printf("Ingrese el costo por hora: ");
    scanf("%f", &tarifa);

    costo = horas * tarifa;

    printf("\n--- RESUMEN ---\n");
    printf("Horas trabajadas: %.2f\n", horas);
    printf("Tarifa por hora: S/ %.2f\n", tarifa);
    printf("Costo total: S/ %.2f\n", costo);

    return 0;
}
```



Figura 1: Resultado del Ejercicio 1

Ejercicio 2. Registro de una observación

Procedimiento: Se utilizan tipos de datos apropiados: entero para el identificador y la edad, float para el promedio, char para la categoría y un entero para representar el estado de validez.

```c
#include <stdio.h>

int main(void) {
    int identificador, edad, valido;
    float promedio;
    char categoria;

    printf("Identificador: ");
    scanf("%d", &identificador);

    printf("Edad: ");
    scanf("%d", &edad);

    printf("Valor promedio: ");
    scanf("%f", &promedio);

    printf("Categoria (una letra): ");
    scanf(" %c", &categoria);

    printf("Registro valido? (1=Si, 0=No): ");
    scanf("%d", &valido);

    printf("\n--- DATOS DE LA OBSERVACION ---\n");
    printf("Identificador: %d\n", identificador);
    printf("Edad: %d\n", edad);
    printf("Promedio: %.2f\n", promedio);
    printf("Categoria: %c\n", categoria);
    printf("Estado: %s\n", valido == 1 ? "Valido" : "No valido");

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Identificador: C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Edad: Valor promedio: Categoria (una letra): Registro valido? (1=Si, 0=No):
--- DATOS DE LA OBSERVACION ---
Identificador: 16
Edad: 346028920
Promedio: 0.00
Categoria: C
Estado: No valido
PS C:\msys64\ucrt64\bin>
```

Figura 2: Resultado del Ejercicio 2

Ejercicio 3. Conversión de decimal a entero

Procedimiento: La conversión explícita mediante `(int)` elimina la parte decimal, sin redondear. La pérdida se calcula restando el entero convertido al valor original.

```c
#include <stdio.h>

int main(void) {
    float medicion, perdida;
    int valorEntero;

    printf("Ingrese la medicion decimal: ");
    scanf("%f", &medicion);

    valorEntero = (int)medicion;
    perdida = medicion - valorEntero;

```

```c
    printf("Valor original: %.2f\n", medicion);
    printf("Valor entero: %d\n", valorEntero);
    printf("Parte decimal perdida: %.2f\n", perdida);

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Ingrese la medicion decimal: 5
Valor original: 5.00
Valor entero: 5
Parte decimal perdida: 0.00
PS C:\msys64\ucrt64\bin>
```

Figura 3: Resultado del Ejercicio 3

Ejercicio 4. Operaciones aritméticas

Procedimiento: Se leen dos enteros y se calculan suma, diferencia, producto, división entera y residuo. Se comprueba que el segundo número no sea cero antes de dividir.

```c
#include <stdio.h>

int main(void) {
    int a, b;

    printf("Ingrese el primer entero: ");
    scanf("%d", &a);

    printf("Ingrese el segundo entero: ");
    scanf("%d", &b);

    printf("\nSuma: %d\n", a + b);
    printf("Diferencia: %d\n", a - b);
    printf("Producto: %d\n", a * b);

    if (b != 0) {
        printf("Division entera: %d\n", a / b);
        printf("Residuo: %d\n", a % b);
    } else {
        printf("No se puede dividir entre cero.\n");
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Ingrese el primer entero: 5
Ingrese el segundo entero: 6

Suma: 11
Diferencia: -1
Producto: 30
Division entera: 0
Residuo: 5
PS C:\msys64\ucrt64\bin>
```

Figura 4: Resultado del Ejercicio 4

Ejercicio 5. Distribución de registros entre nodos

Procedimiento: La división entera determina los registros asignados a cada nodo y el operador módulo determina los registros sobrantes.

```c
#include <stdio.h>

int main(void) {
    int registros, nodos;

    printf("Numero total de registros: ");
    scanf("%d", &registros);

    printf("Numero de nodos: ");
    scanf("%d", &nodos);

    if (registros >= 0 && nodos > 0) {
        printf("Registros por nodo: %d\n", registros / nodos);
        printf("Registros sin distribuir: %d\n", registros % nodos);
    } else {
        printf("Ingrese registros no negativos y nodos mayores que cero.\n");
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Numero total de registros: 5
Numero de nodos: 9
Registros por nodo: 0
Registros sin distribuir: 5
```

Figura 5: Resultado del Ejercicio 5

Ejercicio 6. Operadores de incremento y asignación

Procedimiento: La variable inicia en 4, luego aumenta en 3 y finalmente se multiplica por 2.

```c
#include <stdio.h>

int main(void) {
    int registros = 4;

    printf("Valor inicial: %d\n", registros);

    registros += 3;
    printf("Despues de incrementar en 3: %d\n", registros);

    registros *= 2;
    printf("Despues de duplicar: %d\n", registros);

    return 0;
}
```

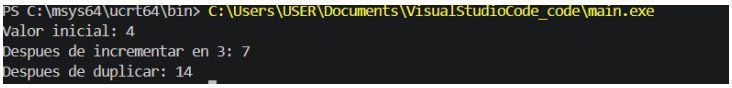


Figura 6: Resultado del Ejercicio 6

Ejercicio 7. Validación de un conjunto de datos

Procedimiento: El conjunto se acepta únicamente si tiene al menos 100 observaciones válidas y un porcentaje de datos completos igual o superior al 70 %. Ambas condiciones deben cumplirse.

```c
#include <stdio.h>

int main(void) {
    int observaciones;
    float porcentaje;

    printf("Numero de observaciones validas: ");
    scanf("%d", &observaciones);

    printf("Porcentaje de datos completos: ");
    scanf("%f", &porcentaje);

    if (observaciones >= 100 && porcentaje >= 70.0 && porcentaje <= 100.0) {
        printf("El conjunto de datos puede ser analizado.\n");
    } else {
        printf("El conjunto de datos no cumple las condiciones.\n");
    }

    return 0;
}
```

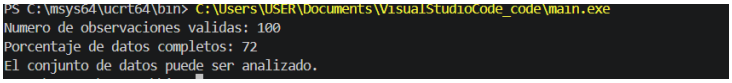


Figura 7: Resultado del Ejercicio 7

Ejercicio 8. Validación de una observación

Procedimiento: Se utiliza el operador lógico AND (`&&`). La observación se incorpora si la edad es como mínimo 18 y el registro está validado.

```c
#include <stdio.h>

int main(void) {
    int edad, validado;

    printf("Ingrese la edad: ");
    scanf("%d", &edad);

    printf("Registro validado? (1=Si, 0=No): ");
    scanf("%d", &validado);

    if (edad >= 18 && validado == 1) {
        printf("La observacion puede incorporarse.\n");
    } else {
        printf("La observacion no puede incorporarse.\n");
```

```
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Ingrese la edad: 19
Registro validado? (1=Si, 0=No): 1
La observacion puede incorporarse.
```

Figura 8: Resultado del Ejercicio 8

Ejercicio 9. Clasificación de temperatura

Procedimiento: Se utilizan condiciones encadenadas para clasificar la temperatura en tres categorías.

```c
#include <stdio.h>

int main(void) {
    float temperatura;

    printf("Ingrese la temperatura en grados Celsius: ");
    scanf("%f", &temperatura);

    if (temperatura < 0) {
        printf("Clasificacion: Congelacion\n");
    } else if (temperatura <= 20) {
        printf("Clasificacion: Frio\n");
    } else {
        printf("Clasificacion: Templado\n");
    }

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Ingrese la temperatura en grados Celsius: 5
Clasificacion: Frio
```

Figura 9: Resultado del Ejercicio 9

Ejercicio 10. Media y observaciones superiores a ella (C++)

Procedimiento: Se almacenan diez mediciones en un arreglo y se calcula su suma. La media se obtiene dividiendo la suma entre diez. Después se cuentan los valores mayores que la media.

```cpp
#include <iostream>
using namespace std;

int main() {
    double datos[10], suma = 0, media;
    int contador = 0;

    for (int i = 0; i < 10; i++) {
        cout << "Ingrese la medicion " << i + 1 << ": ";
        cin >> datos[i];
```

```
        suma += datos[i];
    }

    media = suma / 10;

    for (int i = 0; i < 10; i++) {
        if (datos[i] > media) {
            contador++;
        }
    }

    cout << "Media aritmetica: " << media << endl;
    cout << "Observaciones por encima de la media: " << contador << endl;

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Ingrese la medicion 1: 1
Ingrese la medicion 2: 2
Ingrese la medicion 3: 3
Ingrese la medicion 4: 4
Ingrese la medicion 5: 5
Ingrese la medicion 6: 9
Ingrese la medicion 7: 7
Ingrese la medicion 8: 8
Ingrese la medicion 9: 78
Ingrese la medicion 10: 45
Media aritmetica: 16.2
Observaciones por encima de la media: 2
```

Figura 10: Resultado del Ejercicio 10

Ejercicio 11. Validación de contraseña (C++)

Procedimiento: El ciclo `while` solicita la clave hasta que sea correcta. Se incrementa el contador cada vez que se realiza un intento. Para este ejemplo, la clave correcta se establece en 1234.

```
#include <iostream>
using namespace std;

int main() {
    const int CLAVE_CORRECTA = 1234;
    int clave, intentos = 0;

    cout << "Ingrese la clave numerica: ";
    cin >> clave;
    intentos++;

    while (clave != CLAVE_CORRECTA) {
        cout << "Clave incorrecta. Intente nuevamente.\n";
        cout << "Ingrese la clave numerica: ";
        cin >> clave;
        intentos++;
    }

    cout << "Acceso permitido.\n";
    cout << "Numero de intentos: " << intentos << endl;

    return 0;
}
```

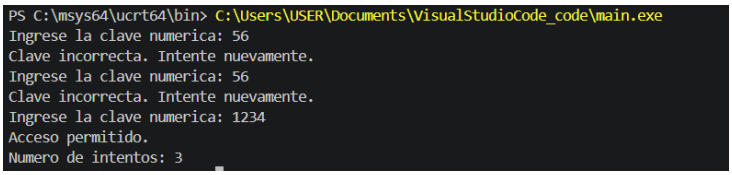


Figura 11: Resultado del Ejercicio 11

Ejercicio 12. Procesamiento de mediciones con break y continue (C++)

Procedimiento: Se leen como máximo diez valores. Los negativos se omiten con `continue`; el valor 999 detiene el ciclo con `break`. Solo se cuentan las mediciones no negativas distintas de 999.

```cpp
#include <iostream>
using namespace std;

int main() {
    double valor;
    int validos = 0;

    for (int i = 0; i < 10; i++) {
        cout << "Ingrese la medicion " << i + 1 << " (999 para terminar): ";
        cin >> valor;

        if (valor == 999) break;

        if (valor < 0) {
            cout << "Valor negativo: se omite.\n";
            continue;
        }

        validos++;
    }

    cout << "Valores validos procesados: " << validos << endl;

    return 0;
}
```

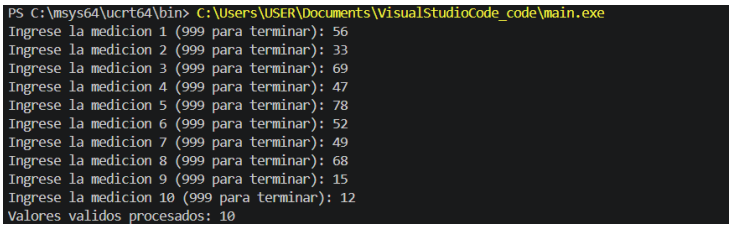


Figura 12: Resultado del Ejercicio 12

Ejercicio 13. Análisis de edades mediante un arreglo

Procedimiento: Se almacenan diez edades. Durante el ingreso se determina la suma, la edad mínima y la máxima. Después se calcula la media y se cuentan las edades superiores a ella.

```c
#include <stdio.h>

int main(void) {
    int edades[10], suma = 0;
    int minima, maxima, superiores = 0;
    float media;

    for (int i = 0; i < 10; i++) {
        printf("Edad del participante %d: ", i + 1);
        scanf("%d", &edades[i]);

        if (i == 0) {
            minima = maxima = edades[i];
        } else {
            if (edades[i] < minima) minima = edades[i];
            if (edades[i] > maxima) maxima = edades[i];
        }

        suma += edades[i];
    }

    media = (float)suma / 10;

    for (int i = 0; i < 10; i++) {
        if (edades[i] > media) superiores++;
    }

    printf("\nEdad minima: %d\n", minima);
    printf("Edad maxima: %d\n", maxima);
    printf("Media de edades: %.2f\n", media);
    printf("Participantes por encima de la media: %d\n", superiores);

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Edad del participante 1: 15
Edad del participante 2: 16
Edad del participante 3: 19
Edad del participante 4: 14
Edad del participante 5: 17
Edad del participante 6: 13
Edad del participante 7: 12
Edad del participante 8: 18
Edad del participante 9: 14
Edad del participante 10: 16

Edad minima: 12
Edad maxima: 19
Media de edades: 15.40
Participantes por encima de la media: 5
```

Figura 13: Resultado del Ejercicio 13

Ejercicio 14. Comparación de complejidad $O(n)$ y $O(n^2)$

Procedimiento: El primer procedimiento ejecuta una operación por cada elemento (n operaciones). El segundo ejecuta dos ciclos anidados de n iteraciones (n^2 operaciones).

```c
#include <stdio.h>

int main(void) {
    int n;
    long long operacionesLineales = 0;
```

```c
    long long operacionesCuadraticas = 0;

    printf("Ingrese el numero de elementos: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("n debe ser un entero positivo.\n");
        return 1;
    }

    /* Procedimiento 1: un recorrido */
    for (int i = 0; i < n; i++) {
        operacionesLineales++;
    }

    /* Procedimiento 2: dos ciclos anidados */
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            operacionesCuadraticas++;
        }
    }

    printf("Operaciones del primer procedimiento: %lld\n",
        operacionesLineales);
    printf("Complejidad aproximada: O(n)\n");

    printf("Operaciones del segundo procedimiento: %lld\n",
        operacionesCuadraticas);
    printf("Complejidad aproximada: O(n^2)\n");

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> C:\Users\USER\Documents\VisualStudioCode_code\main.exe
Ingrese el numero de elementos: 4
Operaciones del primer procedimiento: 4
Complejidad aproximada: O(n)
Operaciones del segundo procedimiento: 16
Complejidad aproximada: O(n^2)
```

Figura 14: Resultado del Ejercicio 14

Ejercicio 15. Funciones para promedio y conteo

Procedimiento: La función `calcular_promedio` recibe tres mediciones y retorna su media. La función `contar_superiores` recibe las mediciones y el promedio, y retorna cuántas superan dicho promedio.

```c
#include <stdio.h>

float calcular_promedio(float a, float b, float c) {
    return (a + b + c) / 3.0f;
}

int contar_superiores(float a, float b, float c, float promedio) {
    int contador = 0;
    if (a > promedio) contador++;
    if (b > promedio) contador++;
    if (c > promedio) contador++;
    return contador;
}
```

```c

int main(void) {
    float m1, m2, m3, promedio;
    int cantidad;

    printf("Ingrese la primera medicion: ");
    scanf("%f", &m1);

    printf("Ingrese la segunda medicion: ");
    scanf("%f", &m2);

    printf("Ingrese la tercera medicion: ");
    scanf("%f", &m3);

    promedio = calcular_promedio(m1, m2, m3);
    cantidad = contar_superiores(m1, m2, m3, promedio);

    printf("Promedio: %.2f\n", promedio);
    printf("Mediciones por encima del promedio: %d\n", cantidad);

    return 0;
}
```

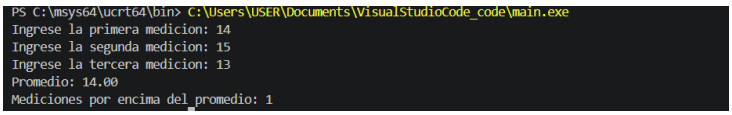


Figura 15: Resultado del Ejercicio 15

Conclusión

Los ejercicios compilados en este informe abarcan los conceptos esenciales del paradigma de programación estructurada: control de flujo mediante estructuras condicionales y bucles, tipos abstractos de datos primitivos, procesamiento secuencial y matricial con arreglos, funciones como unidades modulares y principios básicos sobre el comportamiento temporal de los algoritmos ($O(n)$ y $O(n^2)$).